

Section 2: System Design Fundamentals

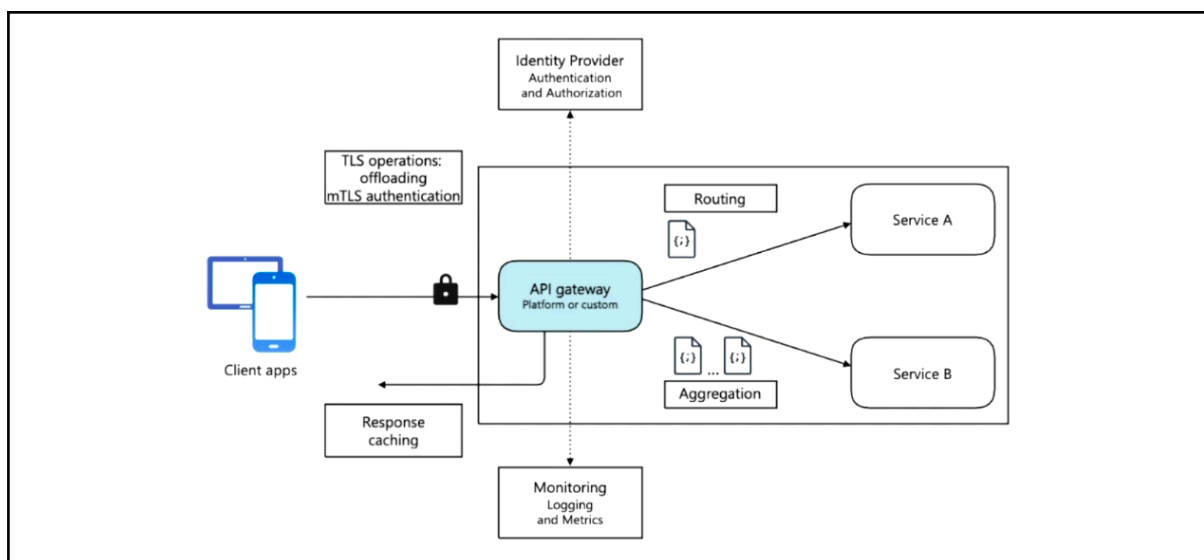
API Gateway

In this session, we will explore how API gateways work, their role in modern system architecture and the key benefits they provide, such as security, rate emitting, caching, etc. API gateways are an essential component in managing and securing APIs, especially in microservices architectures.

By the end of this session, you will have a clear understanding of why API gateways are important, how they function, and when to use them in our system.

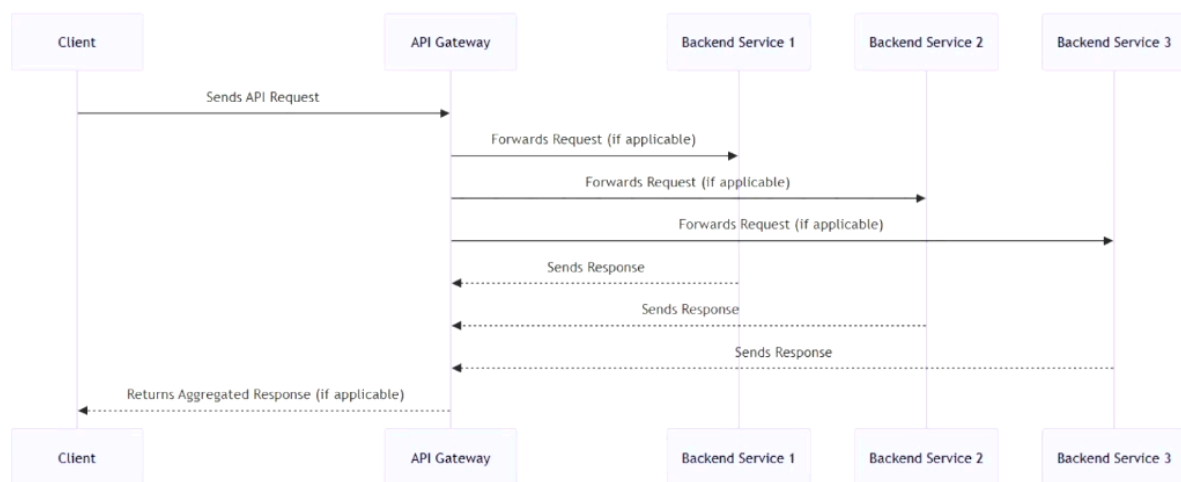
What is API Gateway?

In modern system architecture APIs act as the bridge between clients and servers. Applications like web applications, mobile applications, and third-party integrations with backend services all depend on APIs. These APIs enable seamless communication, but directly exposing the back-end services or APIs to the clients can create several challenges like security risk, performance bottlenecks, and the management overhead that comes with it. This is where an API gateway comes in.



API gateway serves as a centralized entry point for all the API requests, acting as an intermediary between clients and backend-services. It is responsible for essential functions like authentication, request routing, caching and enforcing security in our overall design. This gateway also simplifies API management, enhances security, and optimises performance, making it a crucial component in scalable and secure design.

How API Gateways Work?



An API gateway functions as a reverse proxy, meaning that it sits between the clients and the backend-services. Instead of clients communicating directly with the multiple services that are there on our server side, all the requests first go to the API gateway.

1. When a client makes an API request, the gateway receives it, processes it, and determines which backend-service should handle the request.
2. It then forwards the requests to the appropriate service and returns the response to the client. Beyond the simple request forwarding, API gateway also provides critical functionalities at its end, such as request transformation, meaning modifying the request format to match the backend service expectation.
3. In some cases, it does authentication to verify the user's identity. It can do rate limiting to prevent API abuse. And it can also do the response handling to send the standard responses back to the users.

By centralizing all these tasks in the API gateway, the API gateway simplifies API management. It improves the overall security and enhances the system performance.

Benefits of Using an API Gateway

The below are some of the benefits that makes an API gateway an essential component in the modern API-driven architecture:

- **Security:** By acting as a protective layer, the API gateway prevents direct exposure of the back-end services, reducing security risks and enforcing authentication and authorization.
- **Rate Limiting:** It helps control API traffic. It prevents abuse and mitigates the DDoS attacks by restricting excessive requests from any single client
- **Load Balancing:** It efficiently distributes the incoming traffic across multiple back-end services to ensure high availability.
- **Caching:** It reduces response time by storing frequently requested data, minimizing unnecessary back-end processing.
- **Request Transformation:** The API gateway can modify request format, headers, and protocols enabling seamless communication between clients and back-end services as they might have different requirements when it comes to the format of the requested response.
- **Logging & Monitoring:** Logging and monitoring provides visibility into API usage, how the API is performing, security threats, and it helps the teams to track issues and optimize their systems effectively.

Security Features in API Gateways

Security is one of the most important responsibilities of an API gateway. It provides multiple layers of protection to safeguard back-end services from threats and unauthorized access.

1. **Authentication and authorization:** It enforces authentication and authorization, ensuring that only valid users or applications can access APIs. They support all industry standard methods like OAuth, JWT, and API keys to verify identities and manage permissions.
2. **DDoS Protection:** API gateway can block excessive requests from malicious sources using rate limiting and traffic analysis, and thus preventing the service disruptions due to DDoS attacks.
3. **Bot Protection:** Bot protection helps filter out the automated traffic by applying rate limiting and implementing CAPTCHA-based verifications, the API gateway can ensure that only legitimate users interact with the system and not just bots continuously accessing our applications.
4. **TLS Termination:** SSL Termination secures the communication between clients and services. By handling SSL encryption and decryption at the API gateway level, we are ensuring that these servers are not burdened by this responsibility of decryption and encryption of all the SSL traffic.

So, with these security features in place, an API gateway acts as a very strong defense mechanism, keeping our API secure while maintaining performance and reliability of our application.

Rate Limiting & Throttling

Rate limiting and throttling are essential mechanisms for providing an API gateway to manage API traffic efficiently and protect the backend services.

- Rate limiting sets a cap on the number of API requests per user, or it could be based on per IP or it could be based on per application. It actually specifies that within a given time frame, such as per second, per minute or per hour, how many requests a particular client can make to an API. This prevents excessive or abusive requests from overwhelming the system and ultimately going down.
- If we talk about throttling, throttling is used to control traffic during the peak loads. Instead of outrightly rejecting the request, we will slow down the responses or queue them and ensure that the system remains stable and available even in case of high traffic. And this capability is also baked inside the API gateway. Together these mechanisms help prevent API abuse. They protect the backend services from unexpected traffic spikes, and it ensures fair usage across all the users.

These functionalities are especially useful in scenarios where multiple clients consume the API at scale, and it allows businesses to maintain reliable performance while safeguarding their critical infrastructures.

Caching For Performance Optimization

Caching is a very powerful technique which is used by API gateways to optimise the performance by reducing response time and also reducing the load on the backend server. Instead of processing every request from scratch, caching actually allows the frequently accessed data to be stored temporarily at the API gateway level. So the responses can be served directly from the API gateway for these frequently requested data, and it will be much faster without repeatedly querying the backend services.

There are different types of caching strategies:

- **In-memory caching:** The data is stored in the fast access memory. (e.g., Redis)
- **Response caching:** Stores the API responses so that all the identical requests can be fulfilled quickly without hitting the backend because the response is already stored at the API gateway level.

- **Edge caching:** Integrates alongside CDNs (Content Delivery Networks) which are servers geographically distributed serving content to the user. It stores the cache data closer to the user geographically and thus significantly improving the performance for the global applications.

So, by implementing caching, API gateway enhances the system scalability. It reduces the server load and it improves the overall user experience, making applications much more efficient and responsive.

API Composition & Aggregation

This API composition & aggregation is a very powerful feature because it will help to simplify the client interaction by combining multiple API calls into a single request. And this is particularly useful in microservices architecture where a single client request may require data from multiple backend services. So instead of forcing the client to make separate requests, for example fetching user detail, their order history, their cart items, and in separate API calls, the API gateway can actually call these 3 services internally from the API gateway level, get the responses, aggregate the responses and return a single consolidated result back to the user.

This approach reduces the network overhead, it minimizes latency, and it improves performance. Particularly for mobile and web applications that need to optimize their API calls, this is very important. By offloading this complexity to API gateway level, the client receives faster responses while backend servers remain modular and independently scalable.

Logging & Monitoring With API Gateways

Logging and monitoring are critical functions of an API gateway. It helps the team track the API performance, errors and identify security threats in real time. API gateways generate detailed logs that capture key data points such as request metadata, response time, error rate, and authentication events. These logs are valuable for debugging issues, conducting security audits, and performing usage analysis to understand the API consumption patterns.

To provide much deeper insights, API gateways support monitoring with tools like Prometheus, Grafana, and other Elasticsearch-related tools, which are very famous nowadays. So it supports the integration with all the tools that are available in the market, and it can easily be used with any of these tools for centralized log aggregation and visualisation.

With robust logging and monitoring in place, teams can predictably detect issues, optimize API performance, and ensure the reliability and security of their system. And that's where the logging and monitoring functionality of API gateways are crucial to system design.

Popular API Gateway Implementations

There are many popular API gateway implementations. Each one offers a unique feature which is suited for different use cases.

→ Open-source solutions:

- ◆ Kong
- ◆ NGINX
- ◆ Traefik

→ Cloud-based solutions:

- ◆ AWS API Gateway
- ◆ Google Apigee
- ◆ Azure API Management

Predominantly choosing the right API gateway depends on factors like scalability, cloud compatibility, and security requirements, but it will vary from application to application. So it is highly recommended to analyze your application and then make a judgement call on which API gateway implementation or application to choose from.

When to Use an API Gateway

API gateway are most beneficial in a complex architecture where managing APIs efficiently is crucial. They are best suited for the microservices architecture where multiple services need to communicate seamlessly while maintaining security and performance. They also work very well for multi-client APIs serving web applications, mobile applications, and IoT applications, single server infrastructure server serving multiple type of applications. This ensures consistent access control request handling for all the applications from the same server infrastructure. Additionally, if your APIs require security enforcement, rate limiting, and monitoring, an API gateway is essential to protect and optimize your services.

However, for simple monolithic applications with minimal API exposure, adding an API gateway may introduce unnecessary complexity. So, on the same lines, low traffic, internal only services, direct service communication is preferable over communicating via the API gateway. So using API gateway is optional in these cases.

So understanding when to use an API gateway will help us in designing efficient, scalable, and secure systems. So the first decision you have to make is understand whether you want to use an API gateway or not. If the answer is yes, then try to understand the requirements of your application. Then look at the various API gateway applications and offerings, compare their features, and then make a judgement call on which API gateway you want to use.

Conclusion & Key Takeaways

- API Gateway is a centralized entry point for managing API traffic.
- Provides security, rate limiting, caching, and logging.
- Useful for microservices & high-traffic applications.
- Choosing the right API Gateway depends on use case & scale.
- What's next:
 - Content Delivery Network (CDN)

API Gateway - Interview Questions

- Basic Questions:
 - What is an API Gateway, and why is it used?
 - How does an API Gateway differ from a Load Balancer?
 - What are the key benefits of using an API Gateway?
- Technical Questions:
 - How does an API Gateway handle authentication and authorization?
 - Explain rate limiting and throttling in API Gateways.
 - What caching strategies can be implemented in an API Gateway?
 - How does an API Gateway improve security against DDoS attacks?
- Scenario-Based Questions:
 - When should you use an API Gateway in a microservices architecture?
 - How would you design an API Gateway for a large-scale system with millions of users?
 - What challenges might arise when implementing an API Gateway, and how would you address them?